

- [JS2-数据类型](#)
 - [3.2 比较](#)
 - [== 比较](#)
 - [=== 比较](#)
 - [Object.is\(\)](#)
 - [总结](#)

JS2-数据类型

3.2 比较

== 比较

相等运算符“==”和恒等运算符相似，但相等运算符的比较并不严格。如果两个操作数不是同一类型，那么相等运算符会尝试进行一些类型转换，然后进行比较。具体见[JS2-数据类型-3.1-隐式类型转换](#)。

	null	undefined	false	"false"	Boolean(false)	[]	[""]	""	String("")	0	Number(0)	"0"	String("0")	[0]	true	"true"	Boolean(true)	1	Number(1)	String("1")	[1]	-1	Number(-1)	String("-1")	[-1]	Infinity	-Infinity	{}	NaN
null																													
undefined																													
false																													
"false"																													
Boolean(false)																													
[]																													
[""]																													
""																													
String("")																													
0																													
Number(0)																													
"0"																													
String("0")																													
[0]																													
true																													
"true"																													
Boolean(true)																													
1																													
Number(1)																													
"1"																													
String("1")																													
[1]																													
-1																													
Number(-1)																													
"-1"																													
String("-1")																													
[-1]																													
Infinity																													
-Infinity																													
{}																													
NaN																													

=== 比较

严格相等运算符“===”首先计算其操作数的值，然后比较这两个值，比较过程中没有任何类型转换，它的比较规则如下：

- **不会进行类型转换**：如果两个操作数的类型不同，直接返回 `false`。
- **如果类型相同，再比较值**：
 - 基本类型：值相同（且不为 NaN）时返回 `true`，否则 `false`。特别注意 `NaN === NaN` 结果为 `false`。
 - 对象：仅当两个操作数引用**同一个对象**时返回 `true`，比较的是引用而非内容。
 - `+0` 与 `-0` 被视作相等（`+0 === -0` 为 `true`）。

	null	undefined	false	"false"	Boolean(false)	[]	[""]	String("")	0	Number(0)	String("0")	[0]	true	"true"	Boolean(true)	1	Number(1)	String("1")	[1]	-1	Number(-1)	String("-1")	[-1]	Infinity	-Infinity	{}	NaN
null	1																										
undefined		1																									
false			1																								
"false"				1																							
Boolean(false)					1																						
[]						1																					
[""]							1																				
String("")								1																			
0									1																		
Number(0)										1																	
"0"											1																
String("0")												1															
[0]													1														
true														1													
"true"															1												
Boolean(true)																1											
1																	1										
Number(1)																		1									
"1"																			1								
String("1")																				1							
[1]																					1						
-1																						1					
Number(-1)																							1				
"-1"																								1			
String("-1")																									1		
[-1]																										1	
Infinity																											1
-Infinity																											
{}																											
NaN																											

Object.is()

`Object.is()`是ECMAScript 6中的一个新方法，它用于比较两个值是否相等。它的语法如下：

```
Object.is(value1, value2)
```

当value1和value2具有相同的值时，`Object.is()`返回true。具体来说：

- 对于基本数据类型，如果它们的值相等，则返回true。例如，`Object.is(5, 5)`返回true。

- 对于对象，仅当它们引用同一个对象时，才返回true。例如，`Object.is({}, {})`返回false，而`Object.is(window, window)`返回true。

总结

`Object.is()`与`===`之间的主要区别在于它们如何处理NaN和-0。

- 对于NaN，`Object.is(NaN, NaN)`返回true，而`NaN === NaN`返回false。
- 对于-0和+0，`Object.is(-0, 0)`返回false，而`-0 === 0`返回true。

`Object.is()`与`==`之间的主要区别在于：

- `==` 运算符在判断相等前对两边的变量（如果它们不是同一类型）进行强制转换（这种行为将`"" == false`判断为true）
- `Object.is` 不会强制转换两边的值。

因此，在比较两个值时，需要根据具体情况来选择使用`==`、`===`还是`Object.is()`。